# HashKey Merchant Documentation (All-in-One)

## Table of Contents


- Overview
- Merchant onboarding
- Authentication & signing
- API reference
- Building a Cart Mandate
- Webhooks
- Appendix


---

## Overview

> A crypto payment gateway for merchants, supporting on-chain payments with stablecoins such as USDC and USDT.

HashKey Merchant is a complete cryptocurrency payment solution that gives merchants secure, efficient on-chain payment capabilities.

Through standardized RESTful APIs, your backend can quickly create payment orders, query payment status, and receive final payment outcomes in real time via webhooks.

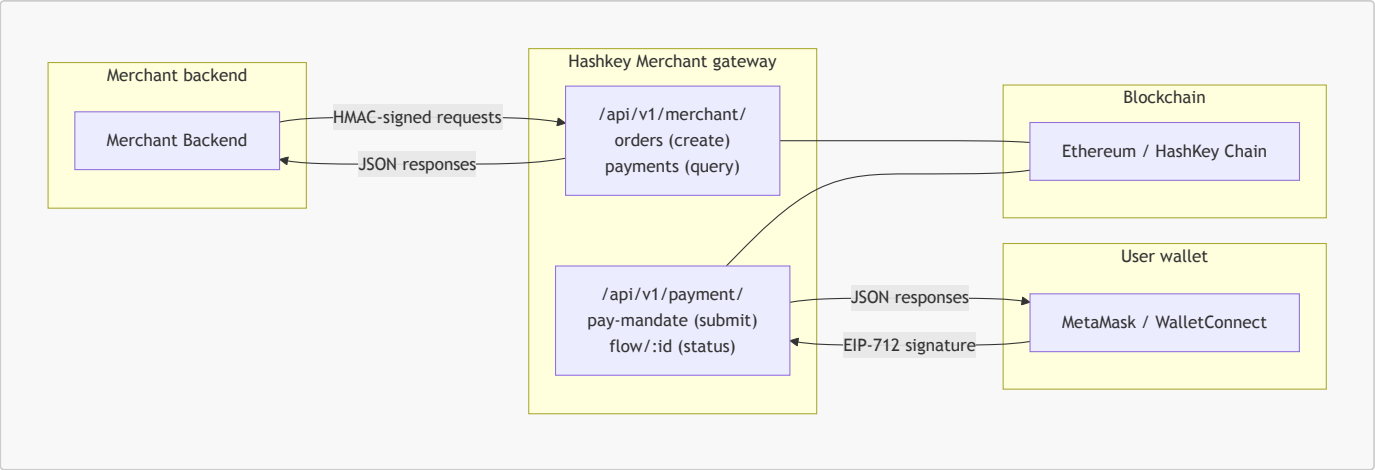
### Key features

| Feature | Description |
|---|---|
| **One-time payment orders** | For e-commerce checkouts, one-off fees, and similar flows - each order maps to a single payment |
| **Reusable payment orders** | For device rental, vending, subscription charges - multiple independent payments under one mandate |
| **Webhook notifications** | Push callbacks after a terminal state, with HMAC-SHA256 verification and up to 6 exponential-backoff retries |
| **Multi-chain & multi-asset** | Ethereum, HashKey Chain, and more; USDC, USDT, and other major stablecoins |
| **HMAC-SHA256 authentication** | All Merchant APIs are protected with HMAC signatures and replay protection |
| **ES256K JWT signing** | Merchant authorization uses secp256k1, aligned with the broader blockchain ecosystem |

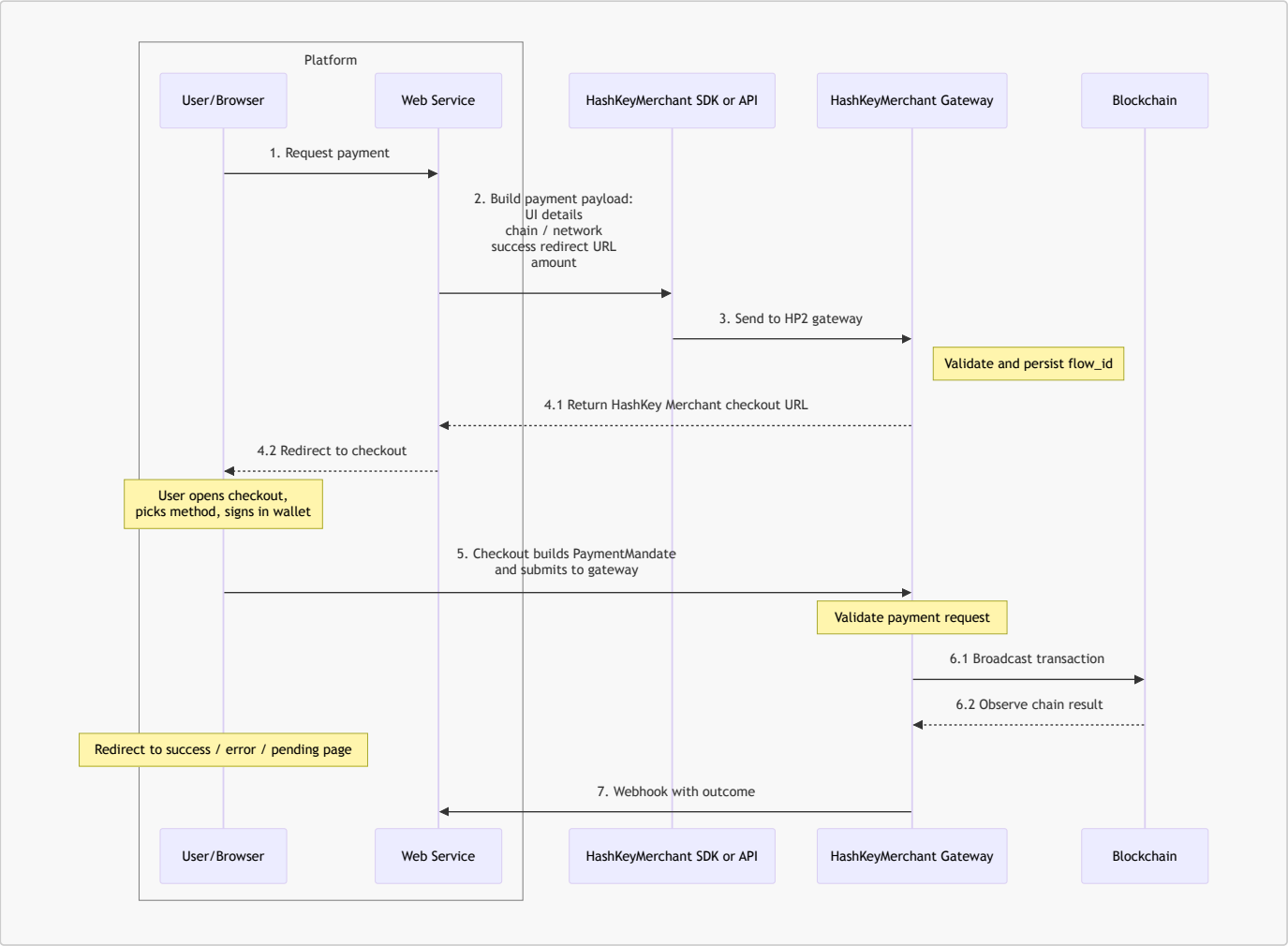
### One-time vs reusable payment orders

| Aspect | One-time order | Reusable order |
|---|---|---|
| **Typical use** | E-commerce, one-off fees, online purchases | Device rental, vending, subscription charges |
| **Payments** | One `cart_mandate_id` -> one payment | One `cart_mandate_id` -> many payments |
| **Create** | `POST /merchant/orders` | `POST /merchant/orders/reusable` |
| **Query** | `GET /merchant/payments` | `GET /merchant/payments/reusable` |
| `cart_expiry` **guidance** | ~2 hours | Cover the full business lifecycle (e.g. 365 days) |

### System architecture




## End-to-end payment flow




1. **Create order** - `POST /api/v1/merchant/orders` to create a Cart Mandate; receive `payment_url` and `flow_id`
2. **Guide the user** - Share `payment_url` (redirect, QR code, email, etc.)
3. **User signs** - User completes EIP-712 authorization in the wallet
4. **Submit payment** - Frontend submits the PaymentMandate
5. **On-chain processing** - Gateway builds and broadcasts the transaction, tracks confirmations
6. **Outcome** - Webhook to your server and/or poll the payment APIs

## Core ID model

| ID | Alias | Producer | Meaning |
|---|---|---|---|
| `cart_mandate_id` | ID1 | Merchant | Order or device identifier - one mandate per order/device authorization |

| ID | Alias | Producer | Meaning |
|---|---|---|---|
| `payment_request_id` | ID2 | Merchant | Payment request id; for one-time orders, pairs 1:1 with `cart_mandate_id` |
| `flow_id` | ID3 | Gateway | Checkout flow id; used in `payment_url` and status queries |
| `payment_mandate_id` | ID4 = ID2 | Frontend | Same as `payment_request_id`; links mandate back to the cart |
| `request_id` | ID5 | Frontend | Unique line-item id generated on the client |

### Cart Mandate & Payment Mandate

**Cart Mandate** and **Payment Mandate** in HashKey Merchant align with the *Verifiable Digital Credentials (VDCs)* model in the **Agent Payments Protocol (AP2)**: standardized, cryptographically bound objects that express merchant and user authorization and the scope of a transaction. See the full definitions, actors, and journeys in the official spec: AP2 specification.

In **HashKey Merchant**, the terms mean the following:

**Cart Mandate** is the **payment request and authorization that the merchant initiates in our system** - the order your backend creates (amount, chain, asset, payee, and related fields), submitted to the gateway together with merchant-side signing such as `merchant_authorization`.

**Payment Mandate** is the **user's authorization to execute the payment / on-chain transfer based on that Cart Mandate** - what the shopper produces at checkout when they sign with their wallet, consenting to move funds under the conditions described in the Cart Mandate.

For fields and flows specific to this product, see Building a Cart Mandate and API reference.

---

## Merchant onboarding

### Steps

1. **Generate a merchant key pair** - secp256k1 (ES256K)
2. **Submit registration details** - organization info, public key, supported chains/tokens
3. **Verify email** - complete signup and bind the organization from the invite link
4. **Create an application** - obtain `app_key` and `app_secret` from the merchant console

### 1. Generate a merchant key pair

You need a **secp256k1 (ES256K)** key pair to sign the `merchant_authorization` JWT.

```
# EC private key (secp256k1)
openssl ecparam -name secp256k1 -genkey -noout -out merchant_private_key.pem

# Export the public key
openssl ec -in merchant_private_key.pem -pubout -out merchant_public_key.pem
```

> [!IMPORTANT] Store the private key only on trusted servers. Never expose it in the browser or commit it to source control.

### 2. Submit registration details

Provide the following to operations so they can create an organization invite:

| Field | Required | Description | Example |
|---|---|---|---|
| `organization_name` | Yes | Organization name | `Acme Pay` |

| Field | Required | Description | Example |
|---|---|---|---|
| `email` | Yes | Admin email | `admin@acme.com` |
| `default_language` | Yes | Default language | `zh-HK` / `en` |
| `public_key` | Yes | Merchant public key (PEM) | `-----BEGIN PUBLIC KEY-----...` |
| `supported_chain_tokens` | Yes | Supported chain/token list (>=1 row) | See below |

If you would like early access to this feature, please send your registration details to hsp_hackathon@hashkey.com.

**Mainnet**

| Network | Chain ID | Token | Contract | Decimals | Stablecoin | Protocol |
|---|---|---|---|---|---|---|
| ethereum | 1 | USDC | `0xa0b86991c6218b36c1d19d4a2e9eb0ce3606eb48` | 6 | Yes | EIP-3009 |
| ethereum | 1 | USDT | `0xdac17f958d2ee523a2206206994597c13d831ec7` | 6 | Yes | Permit2 |
| ethereum | 1 | HSK | `0xe7c6bf469e97eeb0bfb74c8dbff5bd47d4c1c98a` | 18 | No | Permit2 |
| hashkey | 177 | USDC | `0x054ed45810DbBAb8B27668922D110669c9D88D0a` | 6 | Yes | EIP-3009 |
| hashkey | 177 | USDT | `0xF1B50eD67A9e2CC94Ad3c477779E2d4cBfFf9029` | 6 | Yes | Permit2 |

**Testnet**

| Network | Chain ID | Token | Contract | Decimals | Stablecoin | Protocol |
|---|---|---|---|---|---|---|
| sepolia | 11155111 | USDC | `0x1c7D4B196Cb0C7B01d743Fbc6116a902379C7238` | 6 | Yes | EIP-3009 |
| sepolia | 11155111 | USDT | `0xff5588b3b38dff1b4b49bfdcbf985e84d8751a0e` | 6 | Yes | Permit2 |
| sepolia | 11155111 | HSK | `0x31bdac8e4b897e470b70ebe286f94245baa793c2` | 18 | No | Permit2 |
| hashkey-testnet | 133 | USDC | `0x79AEc4EeA31D50792F61D1Ca0733C18c89524C9e` | 6 | Yes | EIP-3009 |
| hashkey-testnet | 133 | USDT | `0x372325443233fEbaC1F6998aC750276468c83CC6` | 6 | Yes | Permit2 |

After the invite is created, the admin receives an email link to finish registration and bind the organization.

### 3. Application credentials

After you create an app in the console, you receive:

| Field | Description | Usage |
|---|---|---|
| `app_key` | Application id | Request header `X-App-Key` |
| `app_secret` | Application secret (**keep private**) | HMAC-SHA256 signing |

You can start calling the Merchant APIs. See Authentication & signing.

---

## Authentication & signing

Hashkey Merchant APIs use two layers:

1. **HMAC-SHA256 request signing** - required on every Merchant API call; proves origin and integrity
2. **ES256K JWT** - the `merchant_authorization` field when creating orders; proves Cart Mandate authenticity

## HMAC-SHA256 request signing

### Required headers

Every `/api/v1/merchant/*` request must include:

| Header | Description | Example |
|---|---|---|
| `X-App-Key` | Application id | `ak_xxxxxxxx` |
| `X-Signature` | HMAC-SHA256 (hex) | `a1b2c3d4e5f6...` |
| `X-Timestamp` | Unix time (seconds) | `1709123456` |
| `X-Nonce` | Anti-replay nonce (UUID or 12-32 hex chars) | `a3f1b2c4d5e6` |

### Algorithm

#### Step 1 - Body hash

- With a JSON body: serialize the payload with **Canonical JSON**, then compute **SHA-256** over the UTF-8 bytes of that string and encode the digest as **lowercase hex** - that value is `bodyHash` (i.e. `bodyHash = hex(SHA256(canonicalJSON(requestBody)))`).
- Without a body (typical GET): `bodyHash` is the empty string `""` (the message format still includes the blank line for it - see examples below).

#### Step 2 - Message string

Join six parts with newline `\n`:

```
message = "{METHOD}\n{PATH}\n{QUERY}\n{bodyHash}\n{timestamp}\n{nonce}"
```

| Part | Description | Example |
|---|---|---|
| `METHOD` | HTTP method (uppercase) | `POST`, `GET` |
| `PATH` | Full path | `/api/v1/merchant/orders` |
| `QUERY` | Query string without `?`, or empty | `cart_mandate_id=ORDER-123` |
| `bodyHash` | SHA256 of body, or empty | `abc123...` |
| `timestamp` | Unix seconds | `1709123456` |
| `nonce` | Random nonce | `a3f1b2c4d5e6` |

#### Step 3 - Sign

```
signature = hex(HMAC-SHA256(app_secret, message))
```

### Examples

#### POST with body

```
POST\n/api/v1/merchant/orders\n\nabc123def456...\n1709123456\na3f1b2c4d5e6
```

#### GET without body

```
GET\n/api/v1/merchant/payments\ncart_mandate_id=ORDER-123\n\n1709123456\na3f1b2c4d5e6
```

#### Security rules

- **Timestamp skew**: +-300 seconds (5 minutes)
- **Nonce uniqueness**: for a given `app_key`, a nonce must not repeat within the 5-minute window

#### Bash (POST)

```bash
#!/bin/bash
APP_KEY="your_app_key"
APP_SECRET="your_app_secret"
TS=$(date +%s)
NONCE=$(openssl rand -hex 16)
PATH_ONLY="/api/v1/merchant/orders"
BODY='{"cart_mandate":{...}}'

BODY_HASH=$(printf "%s" "$BODY" | openssl dgst -sha256 -hex | awk '{print $NF}')
SIGN_STR="POST\n${PATH_ONLY}\n\n${BODY_HASH}\n${TS}\n${NONCE}"
SIG=$(printf "%b" "$SIGN_STR" | openssl dgst -sha256 -hmac "$APP_SECRET" -hex | awk '{print
$NF}')

curl "https://merchant-qa.hashkeymerchant.com${PATH_ONLY}" \
  -X POST \
  -H "Content-Type: application/json" \
  -H "X-App-Key: ${APP_KEY}" \
  -H "X-Timestamp: ${TS}" \
  -H "X-Nonce: ${NONCE}" \
  -H "X-Signature: ${SIG}" \
  -d "$BODY"
```

#### Bash (GET)

```bash
TS=$(date +%s)
NONCE=$(openssl rand -hex 16)
PATH_ONLY="/api/v1/merchant/payments"
QUERY="cart_mandate_id=ORDER-20240301-001"

SIGN_STR="GET\n${PATH_ONLY}\n${QUERY}\n\n${TS}\n${NONCE}"
SIG=$(printf "%b" "$SIGN_STR" | openssl dgst -sha256 -hmac "$APP_SECRET" -hex | awk '{print
$NF}')

curl "https://merchant-qa.hashkeymerchant.com${PATH_ONLY}?${QUERY}" \
  -H "X-App-Key: ${APP_KEY}" \
  -H "X-Timestamp: ${TS}" \
  -H "X-Nonce: ${NONCE}" \
  -H "X-Signature: ${SIG}"
```

### ES256K JWT (`merchant_authorization`)

When creating an order, `cart_mandate.merchant_authorization` must be a JWT signed with your merchant private key.

#### Crypto profile

| Item | Value |
|---|---|
| **Algorithm** | ES256K (ECDSA, secp256k1, SHA-256) |
| **Curve** | secp256k1 (same as Bitcoin/Ethereum) |
| **JWT header** | `{"alg":"ES256K","typ":"JWT"}` |
| **Private key** | PKCS8 (`BEGIN PRIVATE KEY`) or SEC1 (`BEGIN EC PRIVATE KEY`) |

> [!IMPORTANT] JWTs that are not ES256K are rejected.

#### Claims

| Claim | Type | Description |
|---|---|---|
| `iss` | string | Issuer - merchant name |
| `sub` | string | Subject - merchant name |
| `aud` | string | Audience - must be `"HashkeyMerchant"` |
| `iat` | int64 | Issued-at timestamp |
| `exp` | int64 | Expiry (e.g. +1 hour from `iat`) |
| `jti` | string | Unique id, e.g. `JWT-{timestamp}-{random}` |
| `cart_hash` | string | SHA-256 of Cart `contents` (64 hex chars) |

#### Flow

1. Build **Cart contents**
2. **Canonical JSON** -> SHA-256 -> `cart_hash` (see Building a Cart Mandate)
3. **Sign JWT** with ES256K
4. Put the compact JWT into `cart_mandate.merchant_authorization`

#### Key generation

```
openssl ecparam -name secp256k1 -genkey -noout -out merchant_private_key.pem
openssl ec -in merchant_private_key.pem -pubout -out merchant_public_key.pem
```

---

## API reference

Merchant API base path: `https://{host}/api/v1`

All `/merchant/*` routes require HMAC authentication.

### Base URLs

QA (testnet + mainnet tokens): `https://merchant-qa.hashkeymerchant.com`

Staging (mainnet tokens only): `https://merchant-stg.hashkeymerchant.com`

Production (mainnet tokens only): `https://merchant.hashkey.com`

### Overview

| Method | Path | Description | Auth |
|---|---|---|---|
| **POST** | `/merchant/orders` | Create one-time payment order | HMAC |

| Method | Path | Description | Auth |
|---|---|---|---|
| **POST** | `/merchant/orders/reusable` | Create reusable payment order | HMAC |
| **GET** | `/merchant/payments` | Query one-time payments | HMAC |
| **GET** | `/merchant/payments/reusable` | Query reusable payments | HMAC |

### Create one-time order

`POST /merchant/orders`

> Creates a one-time order and returns a checkout URL. The same `cart_mandate_id` cannot be used twice.

**Auth**: HMAC (`X-App-Key`, `X-Signature`, `X-Timestamp`, `X-Nonce`)

**Request body**

```
{
  "cart_mandate": {
    "contents": {
      "id": "ORDER-20240301-001",
      "user_cart_confirmation_required": true,
      "payment_request": {
        "method_data": [
          {
            "supported_methods": "https://www.x402.org/",
            "data": {
              "x402Version": 2,
              "network": "sepolia",
              "chain_id": 11155111,
              "contract_address": "0x1c7D4B196Cb0C7B01d743Fbc6116a902379C7238",
              "pay_to": "0x99c12865b7e4e0cb8a708b448909b76aa1729afd",
              "coin": "USDC"
            }
          }
        ],
        "details": {
          "id": "PAY-REQ-20240301-001",
          "display_items": [
            {"label": "Item A", "amount": {"currency": "USD", "value": "10.00"}},
            {"label": "Item B", "amount": {"currency": "USD", "value": "5.00"}}
          ],
          "total": {"label": "Total", "amount": {"currency": "USD", "value": "15.00"}}
        }
      },
      "cart_expiry": "2024-03-01T12:00:00Z",
      "merchant_name": "My Store"
    },
    "merchant_authorization": "eyJhbGciOiJFUzI1NksiLCJ0eXAiOiJKV1QifQ..."
  },
  "redirect_url": "https://yoursite.com/payment/callback"
}
```

**Field reference**

| Path | Type | Required | Description |
|---|---|---|---|
| `cart_mandate.contents.id` | string | Yes | Order id (`cart_mandate_id`, ID1) |
| `cart_mandate.contents.user_cart_confirmation_required` | bool | Yes | Require user confirmation |

| Path | Type | Required | Description |
|---|---|---|---|
| `cart_mandate.contents.payment_request.method_data` | array | Yes | Payment methods |
| `method_data[].supported_methods` | string | Yes | Currently `"https://www.x402.org/"` |
| `method_data[].data.x402Version` | int | Yes | Must be `2` |
| `method_data[].data.network` | string | Yes | Network name (e.g. `sepolia`) |
| `method_data[].data.chain_id` | int | Yes | Chain id |
| `method_data[].data.contract_address` | string | Yes | Token contract |
| `method_data[].data.pay_to` | string | Yes | Payee address |
| `method_data[].data.coin` | string | Yes | Symbol (e.g. `USDC`) |
| `details.id` | string | Yes | `payment_request_id` (ID2) |
| `details.display_items` | array | No | Line items |
| `details.total` | object | Yes | Total (`label` + `amount`) |
| `contents.cart_expiry` | string | Yes | RFC 3339 expiry (~2h typical) |
| `contents.merchant_name` | string | Yes | Merchant display name |
| `merchant_authorization` | string | Yes | ES256K JWT - Authentication |
| `redirect_url` | string | No | Post-payment redirect |

**Success response**

```
{
  "code": 0,
  "msg": "success",
  "data": {
    "payment_request_id": "PAY-REQ-20240301-001",
    "payment_url": "https://pay.hashkey.com/flow/xxx",
    "multi_pay": false
  }
}
```

| Field | Type | Description |
|---|---|---|
| `payment_request_id` | string | ID2 |
| `payment_url` | string | Checkout URL for the payer |
| `multi_pay` | bool | Always `false` for one-time orders |

> [!NOTE] Duplicate `cart_mandate_id` returns `40001`. For multiple charges on the same device/sku, use the reusable order API.

### Create reusable order

`POST /merchant/orders/reusable`

> Same `cart_mandate_id` can be paid multiple times over its lifetime.

**Auth**: HMAC

**Body / response**: Same shape as one-time, except `multi_pay` is `true`.

```
{
  "code": 0,
  "msg": "success",
  "data": {
    "payment_request_id": "PAY-REQ-001",
    "payment_url": "https://pay.hashkey.com/flow/xxx",
    "multi_pay": true
  }
}
```

> [!IMPORTANT] Set `cart_expiry` to cover the full lifecycle (e.g. rental period). Too short blocks future payments.

### Query one-time payments

`GET /merchant/payments`

> Exactly **one** of the query parameters below.

**Auth**: HMAC

#### Query parameters

| Name | In | Type | Description |
|---|---|---|---|
| `cart_mandate_id` | query | string | By ID1 - returns an array |
| `payment_request_id` | query | string | By ID2 - single object |
| `flow_id` | query | string | By ID3 - single object |

#### Examples

```
GET /api/v1/merchant/payments?cart_mandate_id=ORDER-20240301-001
GET /api/v1/merchant/payments?payment_request_id=PAY-REQ-20240301-001
GET /api/v1/merchant/payments?flow_id=b660fdc3-ac04-437f-921f-efbfb8d089f7
```

#### Sample (`cart_mandate_id`)

```
{
  "code": 0,
  "msg": "success",
  "data": [
    {
      "payment_request_id": "PAY-REQ-20240301-001",
      "request_id": "req_20240301_abc123",
      "token_address": "0x1c7D4B196Cb0C7B01d743Fbc6116a902379C7238",
      "flow_id": "b660fdc3-ac04-437f-921f-efbfb8d089f7",
      "app_key": "ak_xxx",
      "amount": "15000000",
      "usd_amount": "15.00",
      "token": "USDC",
      "chain": "eip155:11155111",
      "network": "sepolia",
      "extra_protocol": "eip3009",
      "status": "payment-successful",
      "payer_address": "0x1234...",
      "to_pay_address": "0x99c1...",
      "risk_level": "Low",
```

```
        "tx_signature": "0xabcd...",
        "broadcast_at": "2024-03-01T10:00:15Z",
        "gas_limit": 150000,
        "gas_fee": "0.001",
        "service_fee_rate": "0.0000",
        "service_fee_type": "free",
        "deadline_time": "2024-03-01T12:00:00Z",
        "created_at": "2024-03-01T10:00:00Z",
        "updated_at": "2024-03-01T10:01:30Z",
        "completed_at": "2024-03-01T10:01:30Z"
      }
    ]
  }
```

Query reusable payments

`GET /merchant/payments/reusable`

> Paginated list or single row. **One** filter at a time.

**Auth**: HMAC

**Query parameters**

| Name | Type | Default | Description |
|---|---|---|---|
| `cart_mandate_id` | string | - | Page by ID1 |
| `flow_id` | string | - | Page by ID3 |
| `request_id` | string | - | Single row by ID5 |
| `page` | int | `1` | Page number |
| `page_size` | int | `20` | Page size |

**Examples**

```
GET /api/v1/merchant/payments/reusable?cart_mandate_id=DEVICE-001&page=1&page_size=20
GET /api/v1/merchant/payments/reusable?flow_id=b660fdc3-ac04-437f-921f-
efbfb8d089f7&page=1&page_size=20
GET /api/v1/merchant/payments/reusable?request_id=req_20240301_abc123
```

**Paged response**

```
{
  "code": 0,
  "msg": "success",
  "data": {
    "list": [
      {
        "payment_request_id": "pay_req_002",
        "request_id": "pay_002",
        "token_address": "0x1c7D4B196Cb0C7B01d743Fbc6116a902379C7238",
        "flow_id": "b660fdc3-ac04-437f-921f-efbfb8d089f7",
        "app_key": "ak_xxx",
        "amount": "1000000",
        "usd_amount": "1.00",
        "token": "USDC",
        "chain": "eip155:11155111",
```

```
          "network": "sepolia",
          "status": "payment-successful",
          "payer_address": "0x1234...",
          "to_pay_address": "0x99c1...",
          "tx_signature": "0xabcd...",
          "created_at": "2024-03-01T11:00:00Z",
          "completed_at": "2024-03-01T11:01:30Z"
        }
      ],
      "total": 15,
      "page": 1,
      "page_size": 20
    }
  }
```

| Field | Type | Description |
|---|---|---|
| `list` | array | Rows, newest `created_at` first |
| `total` | int | Total count |
| `page` | int | Current page |
| `page_size` | int | Page size |

### Payment record fields

Returned on all payment queries (`PaymentItemResponse`):

| Field | Type | Description |
|---|---|---|
| `payment_request_id` | string | ID2 |
| `request_id` | string | ID5 |
| `token_address` | string | Token contract |
| `flow_id` | string | ID3 |
| `app_key` | string | Application id |
| `amount` | string | Amount in smallest units (e.g. USDC 6 decimals) |
| `usd_amount` | string | USD notionals |
| `token` | string | Symbol |
| `chain` | string | CAIP-2 (e.g. `eip155:11155111`) |
| `network` | string | Network name |
| `extra_protocol` | string | `eip3009` / `permit2` |
| `status` | string | See payment state machine |
| `status_reason` | string? | Failure reason |
| `payer_address` | string | Payer |
| `to_pay_address` | string | Payee |
| `risk_level` | string | AML risk |
| `tx_signature` | string | Tx hash when successful |
| `broadcast_at` | string? | First broadcast time |
| `gas_limit` | int | Gas limit |

| Field | Type | Description |
|---|---|---|
| `gas_fee` | string | Gas fee |
| `gas_fee_amount` | string | Gas fee amount |
| `service_fee_rate` | string | Service fee rate |
| `service_fee_type` | string | `free` / `price_include` / `price_extra` |
| `deadline_time` | string | Payment deadline |
| `created_at` | string | Created |
| `updated_at` | string | Updated |
| `completed_at` | string? | Completed when successful |

### Envelope

```
{
  "code": 0,
  "msg": "success",
  "data": { }
}
```

- `code = 0` -> success
- `code != 0` -> error; `msg` is English
- See Error codes

---

## Building a Cart Mandate

The Cart Mandate is the core payload: order metadata, payment methods, line items, and the merchant JWT.

### Full structure

```
{
  "cart_mandate": {
    "contents": {
      "id": "ORDER-001",
      "user_cart_confirmation_required": true,
      "payment_request": {
        "method_data": [{
          "supported_methods": "https://www.x402.org/",
          "data": {
            "x402Version": 2,
            "network": "sepolia",
            "chain_id": 11155111,
            "contract_address": "0x1c7D...",
            "pay_to": "0x99c1...",
            "coin": "USDC"
          }
        }],
        "details": {
          "id": "PAY-REQ-001",
          "display_items": [
            {"label": "Item A", "amount": {"currency": "USD", "value": "10.00"}},
            {"label": "Item B", "amount": {"currency": "USD", "value": "5.00"}}
          ],
          "total": {
            "label": "Total",
```

```
              "amount": {"currency": "USD", "value": "15.00"}
            }
          }
        },
        "cart_expiry": "2024-03-01T12:00:00Z",
        "merchant_name": "My Store"
      },
      "merchant_authorization": "eyJhbG..."
    },
    "redirect_url": "https://yoursite.com/redirect"
  }
```

Field reference

### `contents`

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string | Yes | `cart_mandate_id` (ID1) |
| `user_cart_confirmation_required` | bool | Yes | Require shopper confirmation |
| `payment_request` | object | Yes | Payment details |
| `cart_expiry` | string | Yes | RFC 3339 expiry |
| `merchant_name` | string | Yes | Display name |

### `method_data`

Each `method_data` entry describes one accepted payment method. The gateway currently supports the **x402** protocol; the fields below cover `method_data` and its nested `data` object.

| Field | Type | Required | Description |
|---|---|---|---|
| `supported_methods` | string | Yes | `"https://www.x402.org/"` |
| `data.x402Version` | int | Yes | `2` |
| `data.network` | string | Yes | e.g. `sepolia`, `ethereum` |
| `data.chain_id` | int | Yes | Chain id |
| `data.contract_address` | string | Yes | Token contract |
| `data.pay_to` | string | Yes | Payee |
| `data.coin` | string | Yes | `USDC`, `USDT`, ... |

> [!TIP] Multiple `method_data` entries enable multi-chain / multi-token checkout.

### `details`

| Field | Type | Required | Description |
|---|---|---|---|
| `id` | string | Yes | `payment_request_id` (ID2) |
| `display_items` | array | No | Line items |
| `total` | object | Yes | Total with `label` + `amount` |
| `shipping_options` | array | No | Shipping options |
| `modifiers` | array | No | Modifiers |

### `cart_expiry` guidance

| Scenario | Suggested | Notes |
|---|---|---|
| One-time (e-commerce) | ~2 hours | Covers normal checkout time |
| Reusable (rental, etc.) | >=365 days | Must span the business lifecycle |

> [!IMPORTANT] Short `cart_expiry` expires the mandate. A 2-hour window on a reusable mandate blocks next-day charges.

## Canonical JSON

`cart_hash` uses **Canonical JSON** on `cart_mandate.contents` per RFC 8785.

#### Rules

1. Sort object keys recursively (lexicographic)
2. Compact serialization (no extra whitespace)
3. SHA-256 the string -> 64 hex chars

#### Pseudocode

```
function sortKeys(val) {
  if (val === null || typeof val !== "object") return val;
  if (Array.isArray(val)) return val.map(sortKeys);
  const sorted = {};
  for (const key of Object.keys(val).sort()) {
    sorted[key] = sortKeys(val[key]);
  }
  return sorted;
}

function hashCanonicalJSON(obj) {
  const jsonStr = JSON.stringify(sortKeys(obj));
  return hex(SHA256(jsonStr));
}
```

#### Example

Input:

```
{
  "merchant_name": "My Store",
  "id": "cart-123",
  "cart_expiry": "2024-03-01T12:00:00Z"
}
```

Canonical string:

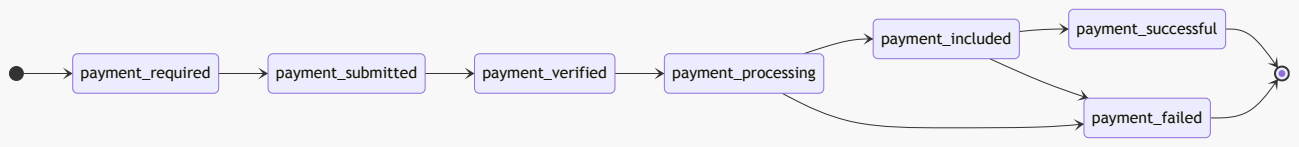
```
{"cart_expiry":"2024-03-01T12:00:00Z","id":"cart-123","merchant_name":"My Store"}
```

Hash it to obtain `cart_hash`.

## Signing pipeline

```
1. Build contents
        |
        v
2. Canonical JSON
        |
        v
3. SHA-256 -> cart_hash
        |
        v
4. JWT claims (iss, sub, aud, iat, exp, jti, cart_hash)
        |
        v
5. ES256K sign -> merchant_authorization
        |
        v
6. Body { cart_mandate, redirect_url? }
        |
        v
7. HMAC -> X-Signature
        |
        v
8. POST /api/v1/merchant/orders
```

Payment state machine




| State | Meaning | Terminal |
|---|---|---|
| `payment-required` | Awaiting payer | No |
| `payment-submitted` | Authorization submitted | No |
| `payment-verified` | Authorization verified | No |
| `payment-processing` | On-chain in flight | No |
| `payment-included` | Included in a block; confirmations pending | No |
| `payment-successful` | Required confirmations met; successful execution | **Yes** |
| `payment-failed` | Failed | **Yes** |

> [!NOTE] Watch `payment-included`, `payment-successful`, and `payment-failed`.
>
> For small amounts or instant fulfillment, `payment-included` is often sufficient; rare reorgs can still fail the tx - otherwise wait for `payment-successful` (often ~20-60 minutes).
>
> `payment-successful` / `payment-failed` are the true terminal states.

---

## Webhooks

When a payment reaches `payment-successful`, `payment-included`, or `payment-failed`, the gateway POSTs to your configured `webhook_url` so you do not need to poll.

### Configuration

`webhook_url` is stored per application (AppCredential) in the merchant console:

1. Sign in -> **Applications**
2. Edit the app
3. Set **Payment webhook URL** (`webhook_url`) - **HTTPS only**

**Requirements**

- Respond with HTTP **2xx within 10 seconds** or the delivery is retried
- If `webhook_url` is empty, no callbacks are sent - poll the APIs instead

### Signature (HMAC-SHA256)

Each callback includes:

```
X-Signature: t=<unix_timestamp>,v1=<hmac_hex>
```

| Part | Meaning |
|---|---|
| `t` | Unix seconds when signed |
| `v1` | HMAC-SHA256 hex digest |

**Payload**

```
message   = timestamp + "." + raw_request_body
signature = hex(HMAC-SHA256(app_secret, message))
```

The key is your application **`app_secret`**.

**Verification checklist**

1. Parse `t` and `v1` from `X-Signature`
2. Ensure `|now - t| <= 300` seconds
3. Recompute HMAC with the raw body
4. Compare with `crypto/subtle` / `hmac.Equal` (constant time)

**Go example**

```go
func verifyWebhookSignature(r *http.Request, rawBody []byte, appSecret string) error {
    sig := r.Header.Get("X-Signature")
    if sig == "" {
        return fmt.Errorf("missing X-Signature header")
    }

    var ts int64
    var received string
    for _, part := range strings.Split(sig, ",") {
        if strings.HasPrefix(part, "t=") {
            ts, _ = strconv.ParseInt(strings.TrimPrefix(part, "t="), 10, 64)
        } else if strings.HasPrefix(part, "v1=") {
            received = strings.TrimPrefix(part, "v1=")
        }
    }

    if math.Abs(float64(time.Now().Unix()-ts)) > 300 {
        return fmt.Errorf("timestamp out of tolerance")
```

```
    }

    msg := fmt.Sprintf("%d.%s", ts, rawBody)
    mac := hmac.New(sha256.New, []byte(appSecret))
    mac.Write([]byte(msg))
    expected := hex.EncodeToString(mac.Sum(nil))

    if !hmac.Equal([]byte(expected), []byte(received)) {
        return fmt.Errorf("signature mismatch")
    }
    return nil
}
```

HTTP request

| Header | Value |
|---|---|
| `Content-Type` | `application/json` |
| `X-Signature` | `t=...,v1=...` |

Payload fields

**Common**

| Field | Type | Description |
|---|---|---|
| `event_type` | string | Always `payment` |
| `payment_request_id` | string | ID2 |
| `request_id` | string | ID5 |
| `cart_mandate_id` | string | ID1 |
| `payer_address` | string | Payer wallet |
| `amount` | string | Smallest units |
| `token` | string | Symbol |
| `token_address` | string | Contract |
| `chain` | string | CAIP-2 |
| `network` | string | Network name |
| `status` | string | `payment-successful` / `payment-failed` / `payment-included` |
| `created_at` | string | RFC 3339 |

**Success extras**

| Field | Type |
|---|---|
| `tx_signature` | string |
| `completed_at` | string |

**Failure extras**

| Field | Type |
|---|---|
| `status_reason` | string |

## Samples

**Success**

```
{
  "event_type": "payment",
  "payment_request_id": "PAY-REQ-20240301-001",
  "request_id": "req_20240301_abc123",
  "cart_mandate_id": "ORDER-20240301-001",
  "payer_address": "0x1234567890abcdef1234567890abcdef12345678",
  "amount": "15000000",
  "token": "USDC",
  "token_address": "0x1c7D4B196Cb0C7B01d743Fbc6116a902379C7238",
  "chain": "eip155:11155111",
  "network": "sepolia",
  "status": "payment-successful",
  "created_at": "2024-03-01T10:00:00Z",
  "tx_signature": "0xabcdef1234567890abcdef1234567890abcdef1234567890abcdef1234567890ab",
  "completed_at": "2024-03-01T10:01:30Z"
}
```

**Failure**

```
{
  "event_type": "payment",
  "payment_request_id": "PAY-REQ-20240301-001",
  "request_id": "req_20240301_def456",
  "cart_mandate_id": "ORDER-20240301-001",
  "payer_address": "0x1234567890abcdef1234567890abcdef12345678",
  "amount": "15000000",
  "token": "USDC",
  "token_address": "0x1c7D4B196Cb0C7B01d743Fbc6116a902379C7238",
  "chain": "eip155:11155111",
  "network": "sepolia",
  "status": "payment-failed",
  "created_at": "2024-03-01T10:00:00Z",
  "status_reason": "Transaction reverted on chain"
}
```

## Your HTTP response

Return **2xx within 10 seconds**:

```
HTTP/1.1 200 OK
Content-Type: application/json

{"code": 0}
```

Only the status code is inspected.

**Best practices**

- Validate business fields (`amount`, `token`, `cart_mandate_id`) before side effects
- Handlers must be **idempotent** - the same `request_id` may arrive more than once due to retries

## Retries

Failed deliveries retry up to **6** times:

| Failure # | Delay |
|---|---|
| 1 | 1 minute |
| 2 | 5 minutes |
| 3 | 15 minutes |
| 4 | 1 hour |
| 5 | 6 hours |
| 6 | 24 hours |

After the final failure the notification is marked `FAILED` and will not retry - query the API for the ground truth.

> [!NOTE] Each `request_id` is delivered at most once at the application layer thanks to internal idempotency.

---

## Appendix

This appendix keeps only content that is not duplicated in previous sections.

### Error codes

**Envelope**

```
{
  "code": 0,
  "msg": "success",
  "data": { }
}
```

- `code = 0` success
- Otherwise `msg` is English

**General (`1xxxx`)**

| Code | HTTP | Meaning | Hint |
|---|---|---|---|
| 10001 | 400 | Invalid parameters | Validate JSON + required fields |
| 10002 | 401 | Unauthorized | Fix HMAC / JWT |
| 10003 | 404 | Not found | Check ids |
| 10004 | 409 | Conflict | Duplicate `payment_request_id`, etc. |
| 10005 | 500 | Internal error | Contact support |
| 10006 | 403 | Forbidden | App permissions |
| 10007 | 429 | Rate limited | Back off |

**Cart Mandate (`4xxxx`)**

| Code | HTTP | Meaning | Hint |
|---|---|---|---|
| 40001 | 400 | Invalid mandate state | Expired or already consumed (one-time) |
| 40002 | 400 | Type mismatch | One-time vs reusable endpoint |

| Code | HTTP | Meaning | Hint |
|---|---|---|---|
| 40003 | 400 | Mandate disabled | Ask admin |
| 40004 | 400 | Mandate revoked | Create a new mandate |

**Application (`5xxxx`)**

| Code | HTTP | Meaning | Hint |
|---|---|---|---|
| 50001 | 409 | App name exists | Pick another name |
| 50002 | 404 | App missing | Check `app_key` |
| 50003 | 403 | Access denied | User <-> app mapping |

**Troubleshooting**

1. Read `code` to pick the bucket (`1` / `4` / `5`)
2. Read English `msg`
3. Correlate with HTTP status
4. Frequent cases: `40001` reused `cart_mandate_id`; `40002` wrong endpoint; `10004` duplicate `payment_request_id`

## Changelog

**v1.1.0 - 2026-03-25**

- Initial release: one-time & reusable orders + payment queries
- HMAC-SHA256 request auth
- ES256K merchant JWT
- Payment webhooks
- Ethereum Sepolia & HashKey Chain testnet support